Análise Experimental da Inferência SepConv para Interpolação de Quadros de Vídeo com Ênfase em Eficiência Computacional e Uso de VRAM

Pedro Henrique Gomes

2026

SENAC Santo Amaro
Bacharelado em Ciência da Computação

Agenda

- Introdução;
- Hipótese científica;
- Objetivos;
- Pesquisa bibliográfica;
- Trabalhos relacionados;
- Fundamentação teórica;
- Proposta de implementação;
- Metodologia experimental;
- Cronograma;

Introdução

- Grande parte dos vídeos atuais ainda opera entre:

$$24 - 30 \ FPS$$

- Displays modernos frequentemente trabalham entre:

$$60Hz \rightarrow 144Hz$$

- Isso gera:
  - baixa fluidez;
  - motion blur;
  - descontinuidade temporal.
- A Video Frame Interpolation (VFI) busca sintetizar quadros intermediários artificialmente.

Problema Central
Modelos modernos de interpolação de quadros possuem elevado custo computacional e alto consumo de memória VRAM.

Desafio
Viabilizar inferência eficiente em GPUs de consumo com aproximadamente 8GB de VRAM.

Hipótese Científica

Hipótese
A separabilidade matemática dos kernels, associada a um pipeline nativo em C++ e CUDA, possui potencial para reduzir a latência e o consumo de memória VRAM em comparação às implementações tradicionais baseadas em frameworks de alto nível.

$$O(N^2) \implies O(2N)$$

- menor custo computacional;
- menor footprint de memória;
- maior viabilidade em hardware de consumo.

Objetivo Principal

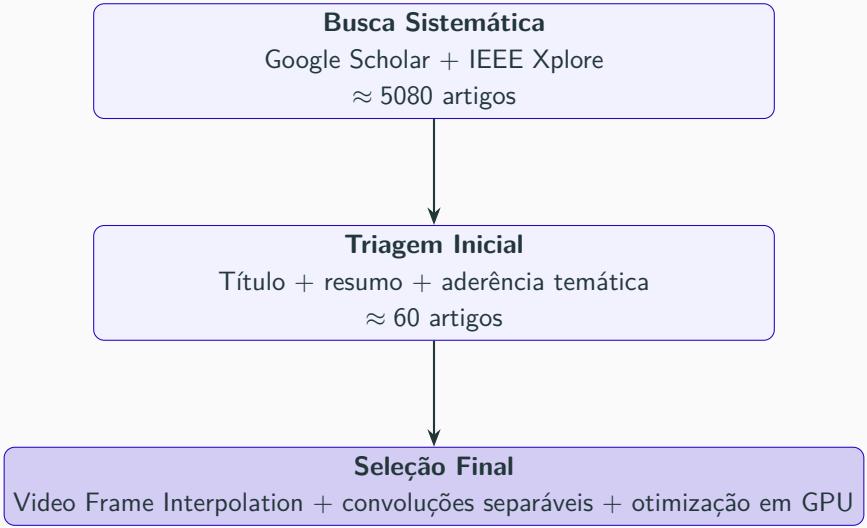
Objetivo Geral
Avaliar experimentalmente os efeitos da convolução separável no desempenho computacional da inferência do modelo SepConv em GPU, utilizando implementação nativa em C++, com ênfase na latência e no consumo de memória VRAM.

Escopo do Trabalho
O foco está exclusivamente na inferência do modelo, utilizando pesos pré-treinados já disponibilizados na literatura.

Objetivos Secundários

1. Desenvolver pipeline em C++ e OpenCV;
2. Integrar o Open Neural Network Exchange Runtime (ONNX Runtime) para inferência do modelo SepConv;
3. Implementar kernels CUDA para convolução separável;
4. Comparar a solução com a baseline em Python e PyTorch;
5. Mensurar:
   - latência;
   - quadros por segundo (FPS);
   - consumo de VRAM.

Metodologia da Pesquisa Bibliográfica



Pesquisa Bibliográfica

Bases Consultadas

- Google Scholar;
- IEEE Xplore.

Palavras-chave

- Video Frame Interpolation;
- Separable Convolution;
- Optical Flow;
- GPU Optimization.

Critérios de Inclusão

- artigos publicados após 2015;
- foco em Video Frame Interpolation;
- inferência em GPU;
- otimização computacional.

Critérios de Exclusão

- baixa aderência técnica;
- dependência extrema de hardware;
- foco exclusivo em treinamento.

Trabalhos Relacionados

Autor	Estratégia	Limitação
Teed e Deng (2020)	Recurrent All-Pairs Field Transforms (RAFT) baseado em fluxo óptico recorrente	Alta precisão, porém elevado custo computacional e grande uso de VRAM
Niklaus et al. (2021)	Convoluções adaptativas separáveis	Dependência de frameworks de alto nível
Wijma et al. (2021)	Convoluções multinível com pirâmides espaciais	Maior robustez, porém aumento do footprint de memória
Este Trabalho	**Inferência nativa em C++ e CUDA**	**Redução de latência e otimização de VRAM**

Fluxo Óptico vs Convolução Adaptativa

RAFT — Teed e Deng (2020)

- estima vetores de movimento;
- utiliza warping;
- múltiplas iterações;
- elevado custo computacional;
- alto consumo de VRAM.

SepConv — Niklaus et al.

- não calcula fluxo explícito;
- movimento implícito no kernel;
- pipeline simplificado;
- menor footprint de memória;
- maior viabilidade computacional.

Diferença Central

O movimento deixa de ser representado por vetores explícitos e passa a ser modelado diretamente pelos pesos do kernel adaptativo.

Fundamentos: Convolução Adaptativa

Convolução Tradicional

$$I'(x,y) = \sum I(x+i, y+j) \cdot K(i,j)$$

- kernel fixo;
- mesma filtragem para toda imagem;
- insuficiente para Video Frame Interpolation.

Convolução Adaptativa
O kernel passa a ser estimado individualmente para cada pixel.

- modela movimento;
- sintetiza o quadro intermediário;
- integra movimento e síntese.

Separabilidade do Kernel

Kernel Tradicional

$$K \in \mathbb{R}^{N \times N}$$

Para:

$$51 \times 51$$

temos:

$$2601$$

coeficientes por pixel.

Kernel Separável

A rede gera:

$$v \in \mathbb{R}^N$$

e

$$h \in \mathbb{R}^N$$

$$K = v \cdot h^T$$

Posto 1 (Rank-1)
O kernel bidimensional pode ser decomposto em dois filtros unidimensionais.

Separabilidade na Prática

Kernel Tradicional 3x3

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Forma Separável

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

- primeiro aplica-se a convolução horizontal;
- depois aplica-se a convolução vertical;
- reduz-se drasticamente o número de multiplicações.

$$O(N^2) \implies O(2N)$$

Método	Operações	Complexidade
Kernel Tradicional	$51 \times 51 = 2601$	$O(N^2)$
Kernel Separável	$51 + 51 = 102$	$O(2N)$

- menos operações matemáticas;
- menor uso de memória;

Fundamentos: Memory Wall

- Em GPUs modernas, o principal gargalo frequentemente não é a computação:

acesso à memória.

- Grandes tensores aumentam:
  - contenção de barramento;
  - tráfego PCI Express (PCIe);
  - pressão sobre VRAM;
  - latência.

- Transferências excessivas entre RAM e VRAM degradam desempenho.

Relação com o Trabalho
A separabilidade reduz o footprint espacial e o tráfego de memória durante a inferência.

Métricas de Avaliação

Peak Signal-to-Noise Ratio (PSNR)

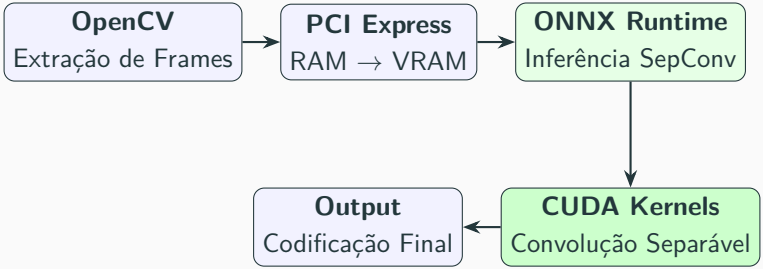
- mede erro pixel a pixel;
- derivado do Mean Squared Error (MSE);
- maior valor implica maior fidelidade visual.

Structural Similarity Index Measure (SSIM)

- avalia similaridade estrutural;
- baseado na percepção humana;
- detecta borramentos e artefatos.

Objetivo
Mensurar o equilíbrio entre qualidade visual e eficiência computacional.

Proposta de Implementação



- utilização de pesos pré-treinados do modelo original;
- inferência mantida na VRAM;
- separação entre rede neural e síntese matemática;
- redução de overhead do PyTorch;
- C++ oferece controle explícito de memória.

Metodologia Experimental

Dataset

- Vimeo-90K;
- cenas com movimento rápido;
- resolução Full HD.

Métricas

- Peak Signal-to-Noise Ratio (PSNR);
- Structural Similarity Index Measure (SSIM);
- quadros por segundo (FPS).

Hardware

- GPU de consumo;
- limite de aproximadamente 8GB de VRAM.

Ferramentas

- NVIDIA Management Library (NVML);
- cudaEventRecord;
- NVIDIA Nsight Systems.

Comparação
A implementação será comparada diretamente com a baseline em Python e PyTorch.

Cronograma do TCC2

Atividade	M1	M2	M3	M4
Revisão Bibliográfica	X			
Pipeline OpenCV	X	X		
Integração ONNX Runtime		X	X	
CUDA Kernels			X	X
Coleta de Métricas			X	X
Redação Final				X

Principais Referências

- Gonzalez e Woods (2009);
- Hennessy e Patterson (2011);
- Niklaus et al. (2017);
- Niklaus et al. (2021);
- Teed e Deng (2020);
- Wang et al. (2004).

Obrigado.